

Junnior

Zaszyfrowana wiadomość : cyberbezpieczeństwo , klucz 21

1. Szyfr Cezara

1. Klucz zależy od konkretnej wiadomości, którą szyfrowałeś — skrypt

sprawdza wszystkie przesunięcia od 0 do 34 i wystarczy znaleźć to, które daje sensowny tekst.
1. Szyfr Cezara jest słaby, ponieważ ma bardzo małą liczbę możliwych kluczy. Można po prostu sprawdzić każdy z nich ręcznie lub skryptem (atak brute-force) i odczytać wiadomość bez znajomości klucza. Nie ma w nim żadnej złożoności matematycznej.
2. Dla polskiego alfabetu (35 znaków) potrzeba maksymalnie **34 próby** (klucz 0 to brak przesunięcia, więc pomijamy go).

2. Funkcje hash

1. Hashowanie jest jednokierunkowe — z hashu nie da się odzyskać oryginalnych danych. Szyfrowanie jest dwukierunkowe — mając klucz, można odszyfrować dane z powrotem do oryginału.
2. MD5 nie jest bezpieczny, ponieważ znaleziono w nim kolizje — można wygenerować dwa różne pliki dające ten sam hash. Jest też na tyle szybki, że ataki brute-force i tablice tęczowe pozwalają łamać hasła hashowane MD5 w krótkim czasie.
3. Bezpieczne algorytmy hashujące to np. **SHA-256** (z rodziny SHA-2) oraz **bcrypt** (szczególnie do haseł).

3. Szyfrowanie symetryczne vs asymetryczne

1. Szyfrowanie symetryczne (np. AES) — szyfrowanie dysku (BitLocker, LUKS), szyfrowanie plików, komunikacja po ustanowieniu sesji w VPN czy HTTPS.
2. Szyfrowanie asymetryczne (np. RSA) — wymiana kluczy przy nawiązywaniu połączenia HTTPS/TLS, podpisy cyfrowe, szyfrowanie e-maili (PGP/GPG), uwierzytelnianie SSH kluczem.

4. Praktyczne zastosowania

1. W przeglądarce — po kliknięciu kłódki przy adresie strony znajdziesz zazwyczaj algorytmy takie jak AES-256-GCM (szyfrowanie symetryczne), ECDHE (wymiana kluczy), RSA lub ECDSA (certyfikat/podpis), SHA-256 (hash). Konkretnie wartości sprawdzisz w szczegółach certyfikatu danej strony.
2. Kryptografia na komputerze/telefonie jest wszędzie: szyfrowanie dysku (BitLocker, FileVault, LUKS), menedżer haseł, Wi-Fi (WPA2/WPA3), komunikatory (Signal, WhatsApp — szyfrowanie end-to-end), VPN, biometria (odcisk palca, Face ID), aktualizacje systemu (podpisy cyfrowe), płatności zbliżeniowe.

Prosty przykład kolizji MD5 — dwa różne ciągi danych dające ten sam hash:

Blok A: d131dd02c5e6eeca4693d9a0698aff95c...

Blok B: d131dd02c5e6eeca4693d9a0698aff95c...

(różnią się kilkoma bajtami, ale MD5 obu = **79054025255fb1a26e4bc422aef54eb4**)

To został udowodniony przez chińskich naukowców (Xiaoyun Wang, 2004). Później w 2012 roku wykorzystano kolizję MD5 w realnym ataku — malware **Flame** użył fałszywego certyfikatu Microsoft wygenerowanego dzięki kolizji MD5, co pozwoliło mu udawać legalną aktualizację Windows.

A jeśli chodzi o szybkość łamania — współczesna karta graficzna potrafi sprawdzić **kilkadziesiąt miliardów** hashy MD5 na sekundę, więc proste hasło hashowane MD5 można złamać w sekundy.

MEDIUM

1. Sprawdzenie i instalacja hashcat na linux - była wersja 6.2
2. Wypakowanie słownika rockyou.txt
3. Utworzenie plików MD5.txt oraz SHA256.txt z hashami do złamania
4. Łamanie haseł

dla md5

b3eeaa937d863a4a0b7c1190e8734815 = monkey1120

76b2ab4cf0d646b748a445647c0feb4d = lemonkingsle

3b76f371357445c27bf732f9c7b73e82 = dontthinktomuch

dla SHA256

2133de89a62c113e414145779fb9ac7efe54d8d947b9f12cd5d9fc3fea01852d:
aiesha724

f36b69b1d1e99f5f81d1f2670ec81807518e259d29f1f7ea1f53eb573470e64f : **TAZ714**

6a420aceb9a8849be48df45842f761c27df37dceb6348b7b565d84d592c481d5 :
06tw03ee88ti

Dlaczego niektóre hashe są łatwiejsze do złamania niż inne?

Zależy od dwóch rzeczy:

Sam algorytm — MD5 jest szybki do obliczenia, więc można sprawdzać miliardy kombinacji na sekundę. SHA-256 jest wolniejszy. Algorytmy jak bcrypt czy Argon2 są celowo bardzo wolne, co sprawia że brute-force trwa lata zamiast sekund.

Samo hasło — hash hasła "password123" złamiesz w sekundy bo jest w każdym słowniku. Hash hasła "Tr@m7!kŁ9ż" będzie praktycznie nie do złamania słownikiem, bo takiej kombinacji nikt nie umieścił w rockyou.txt, a brute-force wszystkich możliwych kombinacji trwałby zbyt długo.

Krótko: słaby algorytm + proste hasło = złamane w sekundy. Silny algorytm + złożone hasło = praktycznie nie do ruszenia.

Jaki słownik użyłeś w Hashcat?
`/usr/share/wordlists/rockyou.txt`

SENIOR

1. Utworzenie nowego pliku `~/hashcat/pesel.txt` w nim zawarte hashe do złamania.

2. Dostosowanie komendy łamiącej hash nie - słownikowy.

słownik rockyou.txt w przypadku liczb nie zadziała. Do tego potrzebny jest atak maskowy.

zamiast:

```
hashcat -m 1400 -a 0 ~/hashcat/sha256.txt  
/usr/share/wordlists/rockyou.txt
```

dajemy:

```
hashcat -m 1400 -a 3 ~/hashcat/pesel.txt ?d?d?d?d?d?d?d?d?d?d?d?d?d
```

gdzie - a 3 to atak maskowy.

"?d" oznaczają cyfry 0-9 do 11 znaków

Numery pesel to :

- 1. 74062086425 - 7bd406ac7f9502c6f.....(skrot)**
- 2. 79092765647 - c41f3fc62b0bbc4e6d00e.....(skrot)**

1. Jakie numery/ciągi znaków kryły się pod hashami niesłownikowymi?
Numery PESEL — 11-cyfrowe ciągi identyfikacyjne.

2. Jakiej maski użyłeś do ataku maskowego?

?d?d?d?d?d?d?d?d?d?d?d — 11 pozycji, każda to cyfra (0-9).

3. Ile czasu zajęło złamanie hashów numerycznych vs słownikowych?
PESEL (SHA-256, brute-force) — szacowany czas to było ok. 48 minut (100 miliardów kombinacji na CPU). MD5 słownikowy (rockyou.txt) — sekundy/minuty, bo hashcat po prostu sprawdzał gotową listę haseł.

Brute-force trwa znacznie dłużej niż atak słownikowy, ale przy krótkich ciągach samych cyfr jest nadal wykonalny.